JULIAN KRAUSS
ITA-09/25 · PROGRAMMIERTECHNIK

VIER GEWINNT

GYM EDITION

C++17 mit Konsole und SFML-2D-Spiel

Projektvorstellung und Live-Code in Visual Studio Code

15 MIN PRÄSENTATION | **15 MIN CODE-DEMO**



© 2026 Julian Krauß · JK AXIS

02 / AGENDA / INHALTSVERZEICHNIS

Was Sie heute erwartet

Eine strukturierte Projektvorstellung und anschließend die Demonstration direkt im Quellcode.

01 **Projektidee & Anforderungen**
Ziel, Spielregeln, flexible Feldgrößen
Folien 03–04

02 **Spielablauf & Objektmodell**
Board, Game, MatchSession und GUI
Folien 05–06

03 **Codeaufbau & Sieglogik**
Header/CPP und vier Prüfungsachsen
Folien 07–08

04 **Persistenz & Komfortbefehle**
Speichern, Laden, Sonderbefehle
Folien 09–10

05 **Tests, Projektstand & Fazit**
CI-Ergebnisse und offene Zielgeräteprüfung
Folien 11–13

06 **Live-Code in Visual Studio Code**
Anschließend 15 Minuten ausführbarer Code
Folie 14

TEIL A · 15 MIN PROJEKTPRÄSENTATION | TEIL B · 15 MIN CODE + DEMO

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

03 / PROJEKTIDEE

Klassisches Spiel. Technische Neuinterpretation.

Das bekannte Spielprinzip bleibt erhalten. Die Umsetzung setzt auf C++17 und objektorientierte Programmierung.

PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

Spielfunktion

Zwei bis vier Personen spielen lokal in der Konsole oder im 2D-Fenster. Vier gleiche Symbole in Folge gewinnen.

Gym-Konzept

Hanteln erscheinen im Terminal als [R] und im 2D-Fenster als animierte Kurzhanteln.



3D-DESIGNSTUDIE · DAS 2D-SPIEL LÄUFT SEPARAT

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

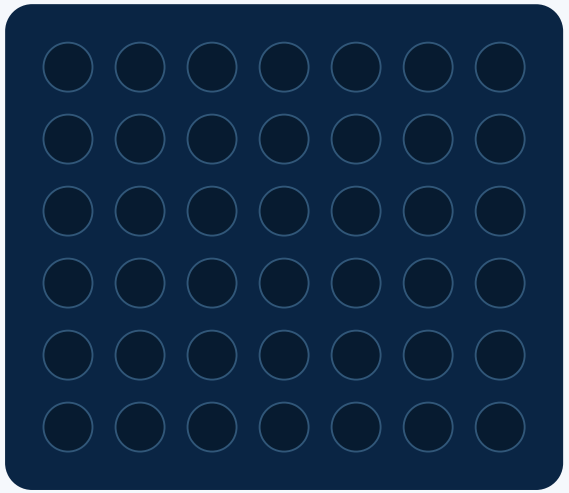
04 / ANFORDERUNGEN

Ein Spielprinzip. Drei Spielfeldgrößen.

Je mehr Personen teilnehmen, desto größer ist das Spielfeld. Die Gewinnbedingung bleibt gleich.

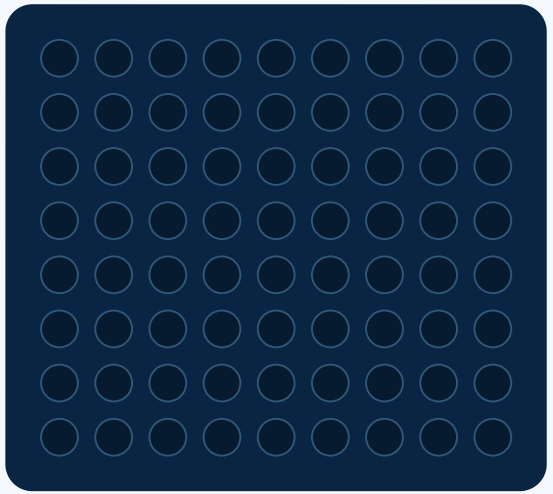
PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

2 SPIELER



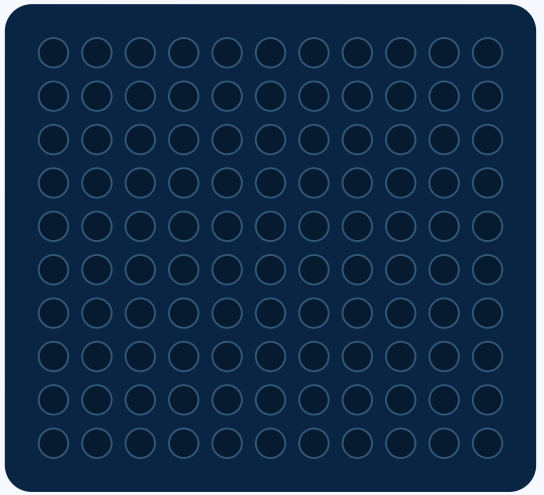
7 × 6

3 SPIELER



9 × 8

4 SPIELER



11 × 10

Gewinnregel 4 in einer Reihe

Ungültige Eingaben werden abgefangen · Nur ein gültiger Spielzug wechselt den Spieler.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

05 / ABLAUF

Vom Startmenü bis zum Ergebnis

Jeder gültige Zug führt durch eine definierte Folge von Zustandsänderungen und Prüfungen.

PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

01

Konfigurieren

Spielerzahl, Namen und GUI-Start bestätigen

02

Stein einwerfen

Spalte validieren und freie Zeile bestimmen

03

Ergebnis prüfen

Sieg, Remis oder nächsten Spieler auswählen

04

Fortsetzen

Nächster Zug, Revanche oder Hauptmenü

Zentrale Regel Eine ungültige Eingabe erzeugt keinen Spielzug und keinen Spielerwechsel.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

06 / ARCHITEKTUR

Gemeinsame Regeln und getrennte Frontends

Board kennt die Regeln. Game und MatchSession steuern ihre jeweilige Partie.

Game + MatchSession

Game steuert die Konsole.
MatchSession hält den GUI-Spielzustand mit Spielern, Zügen, Dateien und Siegen.

Board

Raster, Spielzüge und Gewinnprüfung

Player

Name und eindeutiges Symbol

Move / GymMove

Zeile · Spalte · Spielerindex für die Rückgängig-Historie

Je Controller: 1 Board, 2–4 Player und eine Zughistorie

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

07 / C++-STRUKTUR

Quelltext getrennt nach Verantwortung

Header deklarieren Schnittstellen, CPP-Dateien enthalten die zugehörige Implementierung.

PROJEKTSTRUKTUR

```
include/
  Board.h, Player.h
  Game.h, ConsoleUI.h
  MatchSession.h
  AtomicFile.h
src/
  main.cpp, Game.cpp
  Board.cpp, ConsoleUI.cpp
  MatchSession.cpp
  gui/Main.cpp, GymBeat.h
```

```
Board::Board(int playerCount)
 : rows_(6 + 2 * (playerCount - 2)),
   columns_(7 + 2 * (playerCount - 2)),
   grid_(rows_, std::vector<char>(columns_, '.')) {}
```

EINE REGEL, ZENTRAL DEFINIERT

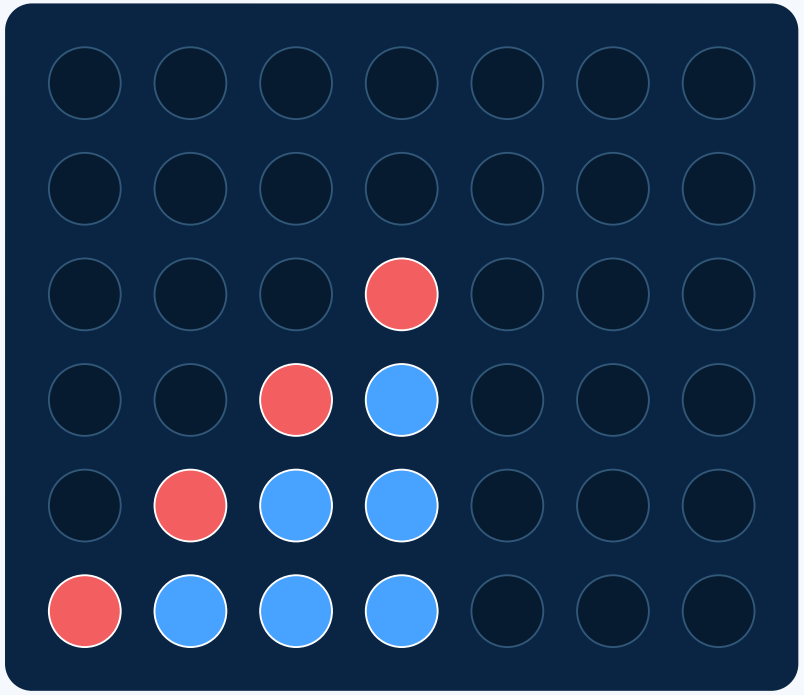
Neue Spielfeldgrößen werden im Konstruktor berechnet. Alle Spielmodi nutzen danach dieselbe Board-Logik.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

08 / ALGORITHMUS

Gewinnprüfung in vier Richtungsachsen

Nach jedem gültigen Zug wird vom letzten Stein aus in beide Richtungen gezählt.



BEISPIEL: DIAGONALE VIERERREIHE

Horizontal · Vertikal · Diagonal ↘ · Diagonal ↙

```
int length = 1
  + countDirection(...,  dr,  dc, symbol)
  + countDirection(..., -dr, -dc, symbol);
if (length >= 4) return true;
```

Nur vier Achsen nötig: jede deckt ihre Gegenrichtung mit ab.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

09 / PERSISTENZ

Spielstände sichern und kontrolliert laden

GYM1 bleibt kompatibel. Beim Schreiben erst vollständig sichern, dann die Zieldatei ersetzen.

SAVEGAME-FORMAT

```
GYM1
2 1 6 7 3
"Julian" R
"Daniel" B
... Zugliste ...
... Spielfeld ...
```

Schematischer Ausschnitt · Formatversion + Zustand

01

Einlesen

Signatur, Dimensionen, Namen und Züge prüfen

02

Rekonstruieren

Temporäres Board für gültige Spielzüge

03

Übernehmen

Erst nach Validierung echten Zustand ersetzen

Laden prüft zuerst einen Kandidaten. Schreibfehler erhalten den getesteten Altbestand.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

10 / FUNKTIONEN

Bedienung und Gym-Identität

Konsole und 2D-Fenster verwenden denselben Regelkern. Zusätze bleiben klar bedienbar.

V 4 **Spaltenvorschau**
nur Anzeige

U **Zug zurücknehmen**
Zughistorie

S **Spielstand speichern**
lokale Datei

F9 **Gym-Beat an/aus**
GUI, Standard aus

I **Partiestatistik**
Füllstand & Spieler

A **Fallanimation an/aus**
Konsole aus, GUI an

Zusätzlich: Revanche, Schullogo und JK-AXIS-Symbol. Das alte 3D-Bild bleibt eine Designstudie.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

11 / QUALITÄTSSICHERUNG

Was ist tatsächlich getestet?

Automatisierte Tests sind von noch offenen manuellen Abnahmetests zu unterscheiden.

AUTOMATISCH VERIFIZIERT

- ✓ Windows und Ubuntu: Debug/Release
- ✓ Board, Konsole und MatchSession
- ✓ GYM1, Revanche und Schreibfehler
- ✓ GUI-Release-Builds auf beiden Systemen
- ✓ Kurzer Linux-Fensterstart über Xvfb

MANUELL NOCH NACHWEISEN

- Lokaler UCRT64-F5-Start
- Mausbedienung und ganze Partie
- Schullogo und tatsächlicher Hörtest
- PowerPoint und Schulbeamer

Nachweis: Commit 02eb79f8, sechs C++-/GUI-Jobs erfolgreich. CI-Snapshot in docs/evidence/.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

12 / PROJEKTSTAND

Implementierter Stand und verbleibende Abnahme

Die Konsole und die zusätzliche SFML-2D-GUI liegen im Entwicklungsbranch. PR #1 ist noch Draft.

IM QUELLCODE VORHANDEN

- Konsole und SFML-2D-Spiel
- Maus/Tastatur und Hantelanimation
- Speichern/Laden und Revanche
- 120-BPM-Beat mit F9
- Schullogo und JK-AXIS-Symbol

OFFEN ODER NICHT IM UMFANG

- Lokaler F5-/Hörtest und Generalprobe
- Vollständige Schulabnahme
- 3D-Spiel
- Online-Multiplayer
- KI-Gegner

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

13 / FAZIT

Gemeinsame Regeln und nachvollziehbare Abläufe

Das Projekt verbindet objektorientierte Modellierung, Regelalgorithmen und sichere Dateiverarbeitung.

PROJEKT | SPIELABLAUF + OOP | ALGORITHMEN | TEST + STATUS | LIVE-CODE

01 Struktur

Game steuert die Konsole, MatchSession die GUI-Partie. Beide nutzen Board.

02 Logik

Ein zentraler Algorithmus prüft jede mögliche Gewinnrichtung.

03 Robustheit

Ungültige Eingaben und geprüfte Schreibfehler werden kontrolliert abgefangen.

NÄCHSTER SCHRITT → QUELLCODE LIVE IM EDITOR

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

14 / LIVE-CODE / AGENDA

Code und beide Spielversionen

15 Minuten für Einstiegspunkte, Board, Gewinnlogik, Dateien und die kurze Vorführung.

00–02	02–05	05–08	08–11	11–15
main.cpp und gui/Main.cpp	**Board-Konstruktor & Spielfeld**	**drop() & hasWon()**	**MatchSession und Dateien**	**GUI und Konsole**

Vorher fertig bauen. demo.ps1 startet zuerst das 2D-Fenster und nach dessen Ende die Konsole.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

15 / RESERVE / RÜCKFRAGEN

Code-Navigation auf einen Blick

Bei Detailfragen lassen sich die Funktionen in den zugehörigen Dateien direkt zeigen.

`main.cpp / gui/Main.cpp`	Zwei Programme: Game starten oder App mit SFML ausführen
`Game.h / MatchSession.h`	Getrennte Controller, Partiedaten und Dateioperationen
`Board.h / Board.cpp`	Raster, nextFreeRow(), drop(), hasWon() und winningCells()
`src/ConsoleUI.cpp`	Terminal-Arena und echte Animationsbilder
`src/gui/GymBeat.h`	Synthetischer 120-BPM-Loop und Wiedergabe
`include/AtomicFile.h`	Temporär schreiben, prüfen und Zieldatei ersetzen

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

16 / RESERVE / FACHFRAGEN

Drei typische Fragen zur Implementierung

Kurz, präzise und mit nachprüfbarem Codebezug antworten.

Warum vier Achsen?

Jede Achse wird in beide Richtungen gezählt. Weitere Richtungen wären redundant.

Warum eine temporäre Board-Instanz beim Laden?

Fehlerhafte Dateien verändern so nicht den laufenden Spielzustand.

Warum Game und MatchSession?

Game steuert die Konsole. MatchSession hält den GUI-Spielzustand. Beide verwenden Board.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg

17 / RESERVE / QUELLEN

Dokumentationsbasis und Status

Aktualisiert am 28.10.2026 anhand von Quellcode, Gesprächsauszug und GitHub-Prüfungen.

Gesamtdokumentation	Aktueller Nachtrag mit ausdrücklich historischem Lern- und Mockup-Teil
Anforderungsanalyse	Pflichtfunktionen, Abnahmekriterien und Projektgrenzen
UML-Klassendiagramm	Gemeinsames Board, Game, MatchSession, App und Datei-Helfer
Quellcode und KI-Protokoll	src/, include/, tests/ sowie Promptregister P-01 bis P-34

KI-Unterstützung offen dokumentiert. CI am Commit 02eb79f8 erfolgreich. Zielgeräteprüfung bleibt offen.

© 2026 Julian Krauß · JK AXIS · Bernd-Blindow-Schulen Ravensburg